5 — Advanced Parametric Study (Scalar + Storage + Time Series)

This module builds an advanced multi-dimensional parametric study with three sweep types: scalar target, storage cost factor, and load time-series scaling.

Learning objectives

- Combine multiple sweep dimensions in one `ParametricStudy`.
- Understand Cartesian case expansion and runtime implications.
- Use `N_HOURS = 7*24*2` (336 time steps) for practical training run time.
- Generate comparative plots and a compact case summary table.

Prerequisites

- You completed modules 1–4.
- You can run SDOM studies with multiprocessing on your platform.

Inputs and scenario used

- Scenario folder: `data/sample_data/`
- Scalar sweep: `GenMix_Target`
- Storage factor sweep: `P_Capex` row in storage data
- Time-series sweep: `load_data` (`Load` column)
- Time horizon in this training script: `N_HOURS = 7*24*2` (336 time steps)

Reference:

- Parametric analysis user guide: https://natlabrockies.github.io/SDOM/user_guide/parametric_analysis.html

Project/file structure for this module

```
training/5_sdom_parametric_advanced/
├── README_m5.md
└── run_m5.py
```

Module suffix and script naming

- Module suffix: `m5`
- Script file: `run_m5.py`

Note on `N_HOURS` in training scripts

`N_HOURS` in this module is a didactic value to keep examples lighter and faster. Real planning runs should use the full year (`8760` hours).

Step-by-step walkthrough

1. Run:

   ```
   python training/5_sdom_parametric_advanced/run_m5.py
   ```

2. Confirm total cases equals the product of sweep lengths.

3. Inspect generated outputs under:

   ```
   results/training/5_sdom_parametric_advanced/
   ```

4. Review the summary CSV with case-level metrics.

Full runnable script

See training/5_sdom_parametric_advanced/run_m5.py.

Inspect advanced parametric API help from Python

This module adds two advanced sweep methods on top of the scalar sweep: `add_storage_factor_sweep` and `add_ts_sweep`. Use `help()` to inspect how SDOM expects the storage parameter name and time-series key.

Run these commands from the repository root with the virtual environment active:

```
python -c "from sdom.parametric import ParametricStudy;
help(ParametricStudy.add_storage_factor_sweep)"
python -c "from sdom.parametric import ParametricStudy;
help(ParametricStudy.add_ts_sweep)"
python -c "from sdom.parametric import ParametricStudy; help(ParametricStudy.run)"
python -c "from sdom.analytic_tools import plot_parametric_results;
help(plot_parametric_results)"
```

Expected output from this environment:

```
Help on function add_storage_factor_sweep in module sdom.parametric.study:

add_storage_factor_sweep(self, param_name: str, factors: list) -> None
   Register a multiplicative storage-parameter sweep.

   Each factor scales the entire ``data["storage_data"].loc[param_name]``
   row (all storage technologies) uniformly.

   Parameters
   ----------
   param_name : str
      Row label in ``data["storage_data"]`` (e.g. ``"P_Capex"``).
   factors : list of float
      Multiplicative factors to apply.
```

```
Help on function add_ts_sweep in module sdom.parametric.study:

add_ts_sweep(self, ts_key: str, factors: list) -> None
   Register a time-series multiplicative sweep.

   Each factor scales the numeric column of ``data[ts_key]``.
   The column name is resolved automatically from
   :data:`sdom.parametric.mutations.TS_KEY_TO_COLUMN`.

   Parameters
   ----------
   ts_key : str
      Key in the SDOM data dict (e.g. ``"load_data"``).
   factors : list of float
      Multiplicative scaling factors.
```

```
Help on function run in module sdom.parametric.study:

run(self) -> List[sdom.results.OptimizationResults]
   Execute all parametric combinations in parallel.
```

```
   Constructs the Cartesian product of all registered sweeps, submits every
   case to a ProcessPoolExecutor, reports progress as jobs complete, exports
   per-case CSVs if output_dir was specified, and writes a summary CSV.
```

```
Help on function plot_parametric_results in module
sdom.analytic_tools._parametric:

plot_parametric_results(study, results, group_by, hue_by=None, facet_by=None,
output_dir=None, max_cases_per_figure=24, plot_per_case=True) -> None
   Generate sensitivity-analysis plots from a completed ParametricStudy run.

   group_by:
      Sweep dimension identifier whose values define the x-axis clusters.
   hue_by:
      Sweep dimension identifier for the bars within each cluster.
   facet_by:
      Sweep dimension identifier to use for faceting.
```

Expected outputs

- Multi-case study outputs for all sweep combinations.
- Parametric plots grouped by `GenMix_Target` and hue by `P_Capex` factor.
- A compact case summary CSV.

How to validate results

- `cases_total = len(genmix) * len(storage_factors) * len(load_factors)`
- Summary CSV exists and has one row per case.
- At least one case solves optimally.

Troubleshooting

- Slow runtime: reduce sweep dimensions for smoke tests.
- Worker failures: inspect case logs in output folder.
- Plot issues: confirm there are successful cases in the result list.

References

- SDOM docs home: https://natlabrockies.github.io/SDOM/index.html
- Parametric user guide: https://natlabrockies.github.io/SDOM/user_guide/parametric_analysis.html
- API core: https://natlabrockies.github.io/SDOM/api/core.html
- Running and outputs: https://natlabrockies.github.io/SDOM/user_guide/running_and_outputs.html